

第四讲

Machine-Level Programming I: Basics

笔记整理自 2026 年 9 月 17 日课程

1. 本讲内容

- Intel x86 处理器与体系结构的演进；
- C、汇编代码与机器代码之间的关系；
- x86-64 的程序员可见状态：程序计数器、寄存器、条件码与内存；
- 寄存器、操作数、数据传送和内存寻址；
- leaq、算术运算、逻辑运算与移位。

本讲使用 **AT&T 汇编语法**：源操作数在前、目的操作数在后，寄存器名前有 %，立即数前有 \$。Intel 官方手册通常使用相反的操作数顺序，阅读时必须先确认语法。

2. x86 处理器与体系结构

2.1. x86 的演进

x86 从 1978 年的 8086 演进至今。其设计强调向后兼容：新处理器仍要支持许多早期软件所依赖的指令和行为。结果是指令数量多、格式不统一、历史遗留特性明显，因此通常被归为复杂指令集计算机（CISC）。不过，实际 Linux 程序只会频繁使用其中一小部分指令。

处理器	年份	晶体管数	意义
8086	1978	约 2.9 万	首款 16 位 Intel 处理器；IBM PC 与 DOS 的基础；1 MiB 地址空间
80386	1985	约 27.5 万	首款 32 位 x86 处理器；形成 IA-32；支持平坦地址空间
Pentium 4E	2004	约 1.25 亿	Intel 首批支持 64 位 x86 的处理器
Core 2	2006	约 2.91 亿	Intel 多核处理器时代的重要起点
Core i7	2008	约 7.31 亿	四核桌面处理器代表

处理器演进并不只是提高时钟频率。后来还加入了多媒体与向量指令、更高效的条件操作、更宽的地址和数据通路以及更多核心。制程尺寸则从数百纳米逐渐缩小到十几纳米乃至更小；所谓制程尺寸可粗略理解为芯片中最窄互连或器件特征的尺度。

2.2. 从 IA-32 到 x86-64

Intel 曾尝试用 IA-64（Itanium）取代 IA-32，但 IA-64 与旧 x86 架构差异很大，兼容和性能表现都不理想。AMD 在 2003 年提出向后兼容的 64 位扩展 x86-64（也称 AMD64）；Intel 随后推出几乎相同的 EM64T。今天，桌面和服务程序通常以 x86-64 模式运行，本课程也只讨论 x86-64。

例：为什么向后兼容重要？假设一家公司有一套十年前编译的财务软件。如果新处理器仍实现旧指令集，这套二进制程序可以直接运行；若处理器采用完全不同的指令集，就必须重新编译、模拟旧机器，甚至重写依赖旧环境的部分。

2.3. ISA 与微体系结构

- **指令集体系结构 (ISA)**：为了编写正确机器代码而必须知道的接口，包括指令语义、寄存器、数据类型和寻址方式等。机器代码是处理器执行的字节序列，汇编代码则是它的文本表示。

- **微体系结构 (microarchitecture)**：ISA 的具体实现方式，例如流水线深度、缓存大小、执行单元数量和核心频率。

同一 ISA 可以有多种微体系结构。两款处理器都能执行同一个 `addq`，说明它们遵循相同的 ISA；一款更快，可能是因为缓存更大或每周期能发射更多指令，而不是 `addq` 的含义发生了变化。常见 ISA 还包括 ARM 和开放的 RISC-V。

课前笔记中提到的 `popcnt` 是一个很好的例子：它统计操作数二进制表示中 1 的个数。例如 `popcnt(0b10110100)=4`。x86 指令很多，但编译普通程序时，真正频繁出现的只是较小子集。

3. 从 C 程序到机器代码

3.1. 程序员可见的机器状态

从汇编程序的视角，处理器和内存可以抽象为以下状态：

- **程序计数器 (PC)**：保存下一条将执行的指令地址；在 x86-64 中称为 `%rip`。
- **整数寄存器组**：保存频繁使用的数据、地址和中间结果。
- **条件码**：记录最近一次算术或逻辑运算的某些性质，例如结果是否为零、是否为负、是否产生进位或溢出；条件跳转会读取这些状态。
- **内存**：按字节编址的数组，保存代码、全局数据、堆和栈。栈还用于支持过程调用。

寄存器与内存的角色不同。寄存器数量少但访问快，名称直接写在指令里；内存容量大，通过地址访问。编译器的重要任务之一，就是决定哪些值暂存在寄存器中，哪些值必须放入内存。

3.2. 编译、汇编与链接

典型构建过程为：

C 源文件 (`.c`) → 编译器 → 汇编文件 (`.s`) → 汇编器 → 目标文件 (`.o`) → 链接器 → 可执行文件

- 编译器把 C 语句转换成汇编指令。使用 `gcc -Og -S sum.c` 可以生成较适合学习和调试的汇编文件。
- 汇编器把每条汇编指令编码成机器字节，生成目标文件。目标文件已经很接近可执行映像，但跨文件引用可能尚未解析。
- 链接器解析文件之间的符号引用，并组合目标文件与库，生成可执行文件。静态库的代码可以在链接时并入；动态库的最终绑定可能延迟到程序装载或运行时。

编译器版本、优化级别和目标平台都会显著改变输出。因此，同一个 C 函数在不同机器上产生的汇编不必逐字相同，只要行为一致即可。

3.3. 一个完整例子：sumstore

```
long plus(long x, long y);
```

```
void sumstore(long x, long y, long *dest) {
    long t = plus(x, y);
    *dest = t;
}
```

一种可能的 x86-64 汇编为：

```
sumstore:
    pushq %rbx
    movq  %rdx, %rbx
    call  plus
    movq  %rax, (%rbx)
    popq  %rbx
    ret
```

这里暂时只需把它理解为：保存 `%rbx`；把指针 `dest` 暂存到 `%rbx`；调用 `plus`；从 `%rax` 取得返回值；把它写到 `dest` 指向的内存；恢复 `%rbx` 并返回。函数参数和寄存器的严格约定将在过程调用一讲中讨论。

真实的 `.s` 文件还包含 `.globl`、`.type`、`.cfi_startproc` 等以句点开头的内容。这些通常是给汇编器、链接器或调试器看的伪指令（directive），不是处理器实际执行的机器指令。

3.4. 机器代码与反汇编

上例中的

```
movq %rax, (%rbx)
```

可编码为 3 个字节 `48 89 03`。其中 `%rax` 保存 `t`，`%rbx` 保存 `dest`，`(%rbx)` 表示地址 `%rbx` 指向的内存位置，因此它对应 C 语句 `*dest = t`。

反汇编器从字节模式重建近似的汇编表示。常用命令是 `objdump -d program`；在 GDB 中可以用 `disassemble function` 查看函数，用 `x/14xb address` 查看从指定地址开始的 14 个字节。

反汇编器并不知道原始 C 类型、变量名或程序意图。只要一段字节被当作可执行代码解释，它就能尝试反汇编；输出是否有意义还取决于起始地址和所选 ISA 是否正确。

4. 汇编语言基础

4.1. 汇编层的数据与操作

汇编语言中的数据类型比 C 简单得多：

- 整数数据通常为 1、2、4 或 8 字节；地址也只是无类型的整数位模式；
- 浮点数据常见为 4、8 或 10 字节；向量数据可更宽；
- 数组和结构体不会作为独立的机器类型出现，它们只是内存中的连续字节；
- 指令本身也是一串字节。

基本操作可以归纳为三类：在寄存器与内存之间传送数据；对寄存器或内存数据做算术/逻辑运算；通过调用、返回和条件/无条件跳转转移控制。

例：C 语言结构体 `struct Point { long x; long y; }`；在汇编看来只是连续的 16 字节。若结构体首地址在 `%rdi` 中，那么 `x` 位于 `(%rdi)`，`y` 位于 `8(%rdi)`；“这是一个 Point”属于编译器和程序员的解释，而不是内存自身携带的标签。

4.2. x86-64 整数寄存器

x86-64 有 16 个通用整数寄存器。寄存器的不同名称可以访问同一寄存器的不同低位部分。

64 位	低 32 位	低 16 位	低 8 位	历史用途或常见角色
<code>%rax</code>	<code>%eax</code>	<code>%ax</code>	<code>%al</code>	累加器；常保存返回值
<code>%rbx</code>	<code>%ebx</code>	<code>%bx</code>	<code>%bl</code>	基址寄存器
<code>%rcx</code>	<code>%ecx</code>	<code>%cx</code>	<code>%cl</code>	计数器
<code>%rdx</code>	<code>%edx</code>	<code>%dx</code>	<code>%dl</code>	数据；也常传递参数
<code>%rsi</code>	<code>%esi</code>	<code>%si</code>	<code>%sil</code>	源索引；常传递参数
<code>%rdi</code>	<code>%edi</code>	<code>%di</code>	<code>%dil</code>	目的索引；常传递参数
<code>%rsp</code>	<code>%esp</code>	<code>%sp</code>	<code>%spl</code>	栈指针
<code>%rbp</code>	<code>%ebp</code>	<code>%bp</code>	<code>%bpl</code>	历史上的帧基址

其余寄存器为 `%r8` 至 `%r15`，相应的低 32 位写作 `%r8d` 至 `%r15d`，低 16 位写作 `%r8w` 至 `%r15w`，低 8 位写作 `%r8b` 至 `%r15b`。`%rsp` 有专门的栈语义，不能像普通索引寄存器一样随意使用。

写入 32 位寄存器会自动把对应 64 位寄存器的高 32 位清零。例如执行 `movl $1, %eax` 后, `%rax` 的值是 64 位的 1。写入 8 位或 16 位子寄存器则不会自动清除其余高位。

4.3. movq 与三类操作数

`movq Source, Dest` 传送 8 字节 (quad word)。操作数可分为:

- **立即数**: 写在指令中的常量, 如 `$0x400`、`-$533`;
- **寄存器**: 如 `%rax`、`%r13`;
- **内存**: 由地址表达式指定的位置, 如 `(%rax)` 表示从 `%rax` 中的地址开始的 8 个字节。

源	目的	汇编	近似的 C 含义
立即数	寄存器	<code>movq \$0x4, %rax</code>	<code>temp = 4</code>
立即数	内存	<code>movq \$-147, (%rax)</code>	<code>*p = -147</code>
寄存器	寄存器	<code>movq %rax, %rdx</code>	<code>b = a</code>
寄存器	内存	<code>movq %rax, (%rdx)</code>	<code>*p = a</code>
内存	寄存器	<code>movq (%rax), %rdx</code>	<code>b = *p</code>

x86 不允许用一条普通 `movq` 完成内存到内存的传送; 必须先读入寄存器, 再写回内存。

例: 若 `%rax=0x100`, 且内存地址 `0x100` 开始的 8 字节表示整数 42, 则 `movq (%rax), %rdx` 执行后 `%rdx=42`。注意 `%rax` 保存的是地址, `(%rax)` 才是该地址处的内容。

4.4. 简单内存寻址

最简单的两种地址形式为:

- (R): 有效地址为 `Reg[R]`, 对应 C 中的指针解引用;
- D(R): 有效地址为 `Reg[R] + D`, 其中 `D` 是常量偏移量。

例如 `movq (%rcx), %rax` 从 `%rcx` 指向的位置读取 8 字节; `movq 8(%rbp), %rdx` 从地址 `%rbp+8` 读取 8 字节。

4.5. swap: 从 C 到汇编

```
void swap(long *xp, long *yp) {
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
swap:
    movq (%rdi), %rax    # t0 = *xp
    movq (%rsi), %rdx    # t1 = *yp
    movq %rdx, (%rdi)    # *xp = t1
    movq %rax, (%rsi)    # *yp = t0
    ret
```

假设 `%rdi=0x120`、`%rsi=0x100`, 初始时 `M[0x120]=123`、`M[0x100]=456`:

步	执行后	<code>%rax</code>	<code>%rdx</code>	<code>M[0x120]</code>	<code>M[0x100]</code>
0	初始状态	—	—	123	456
1	<code>movq (%rdi), %rax</code>	123	—	123	456
2	<code>movq (%rsi), %rdx</code>	123	456	123	456
3	<code>movq %rdx, (%rdi)</code>	123	456	456	456
4	<code>movq %rax, (%rsi)</code>	123	456	456	123

两个临时值都先保存在寄存器中，所以第一次写内存不会丢失原来的 123。

4.6. 完整内存寻址

最一般的地址形式是

$$D(R_b, R_i, S) \rightarrow \text{Mem}[\text{Reg}[R_b] + S \times \text{Reg}[R_i] + D]$$

其中：

- D 是 1、2 或 4 字节编码的常量位移；
- R_b 是基址寄存器；
- R_i 是索引寄存器，但不能是 `%rsp`；
- S 是比例因子，只能取 1、2、4 或 8。

比例因子的取值恰好适合常见元素大小。若 `long a[]` 的首地址在 `%rdi`、下标 `i` 在 `%rsi`，由于每个 `long` 占 8 字节，`a[i]` 的地址就是 `(%rdi,%rsi,8)`。

设 `%rdx=0xf000`、`%rcx=0x0100`：

地址表达式	计算	有效地址
<code>0x8(%rdx)</code>	<code>0xf000 + 0x8</code>	<code>0xf008</code>
<code>(%rdx,%rcx)</code>	<code>0xf000 + 0x100</code>	<code>0xf100</code>
<code>(%rdx,%rcx,4)</code>	<code>0xf000 + 4 * 0x100</code>	<code>0xf400</code>
<code>0x80(,%rdx,2)</code>	<code>2 * 0xf000 + 0x80</code>	<code>0x1e080</code>

5. 算术与逻辑运算

5.1. `leaq`：只计算地址，不访问内存

`leaq Src, Dst` 把地址表达式 `Src` 计算出的**有效地址**写入寄存器 `Dst`，但不会读取该地址处的内存。它有两类常见用途：

- 计算指针，如 C 中的 `p = &x[i]`；
- 利用地址计算硬件完成 $x + ky$ ，其中 $k \in \{1, 2, 4, 8\}$ 。

例如：

```
long m12(long x) {
    return x * 12;
}
```

可编译为：

```
leaq (%rdi,%rdi,2), %rax    # %rax = x + 2x = 3x
salq $2, %rax               # %rax = 3x << 2 = 12x
```

这里 `(%rdi,%rdi,2)` 看起来像内存操作数，但 `leaq` 只取它算出的数值 $3x$ ，不会访问地址 $3x$ 。

5.2. 双操作数指令

AT&T 语法中的一般形式为 `opq Src, Dest`，结果写回 `Dest`。

指令	计算	说明
<code>addq S, D</code>	$D \rightarrow D + S$	加法
<code>subq S, D</code>	$D \rightarrow D - S$	减法；注意操作数顺序
<code>imulq S, D</code>	$D \rightarrow D \times S$	取乘积的低 64 位
<code>salq S, D</code>	$D \rightarrow D \ll S$	左移；也写作 <code>shlq</code>

指令	计算	说明
sarq S, D	$D \rightarrow D \gg S$	算术右移，用符号位填充
shrq S, D	$D \rightarrow D \gg S$	逻辑右移，用 0 填充
xorq S, D	$D \rightarrow D \oplus S$	按位异或
andq S, D	$D \rightarrow D \wedge S$	按位与
orq S, D	$D \rightarrow D \vee S$	按位或

incq、decq、negq、notq 是单操作数指令，分别执行加一、减一、取负和按位取反。

例：算术右移与逻辑右移。用 8 位补码演示，1111 0000 表示有符号数 -16，也可表示无符号数 240。右移两位时，算术右移补 1，得到 1111 1100 (-4)；逻辑右移补 0，得到 0011 1100 (60)。位模式相同，但解释方式决定应选哪种右移。

对固定宽度整数而言，加法、减法、按位操作和低位乘积只处理位模式，通常不需要区分有符号或无符号；差别体现在比较、右移、高位乘积以及如何解释条件码等操作中。

5.3. 算术表达式翻译示例

```
long arith(long x, long y, long z) {
    long t1 = x + y;
    long t2 = z + t1;
    long t3 = x + 4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    return t2 * t5;
}
```

一种编译结果为：

```
arith:
    leaq (%rdi,%rsi), %rax      # t1 = x + y
    addq %rdx, %rax            # t2 = z + t1
    leaq (%rsi,%rsi,2), %rdx    # 3y
    salq $4, %rdx              # t4 = 3y * 16 = 48y
    leaq 4(%rdi,%rdx), %rcx     # t5 = x + 4 + 48y
    imulq %rcx, %rax           # return t2 * t5
    ret
```

这个例子体现了三点：

- 一个寄存器可以在不同时刻保存不同的 C 临时变量，例如 %rdx 先保存参数 z，后保存 t4；
- 编译器不必逐句保留 C 的临时变量，而会直接形成更紧凑的数据流；
- 常数乘法常被拆成 leaq 与移位，因为左移 k 位等价于乘 2^k 。这里 $48y = (3y) \times 16$ ，因此只在最后使用一次真正的乘法指令。

6. 易错点与自测

6.1. 常见易错点

- 混淆 AT&T 与 Intel 语法的源、目的顺序；
- 把 %rax（寄存器中的值）与 (%rax)（该值作为地址所指向的内存）混为一谈；
- 误以为 leaq 一定访问内存；它只计算地址表达式；
- 尝试用一条 movq 在两个内存位置之间直接复制；
- 忘记比例因子只允许 1、2、4、8；
- 把算术右移 sarq 与逻辑右移 shrq 混用；

- 从一段汇编中强行恢复唯一的 C 源码。编译优化会删除变量、合并表达式，因此通常只能恢复等价逻辑。

6.2. 自测题

- 1) 若 `%rax=0x100`、`M[0x100]=7`，说明 `%rax` 与 `(%rax)` 各表示什么。
- 2) 若 `%rbx=0x200`、`%rcx=3`，求 `16(%rbx,%rcx,8)` 的有效地址。
- 3) 写出把内存 `(%rdi)` 的 8 字节复制到内存 `(%rsi)` 所需的两条 `movq`。
- 4) 解释为什么 `leaq (%rdi,%rdi,4), %rax` 可以计算 $5x$ 。
- 5) 对位模式 `0xfffffffffffffffff0`，分别说明 `sarq $2` 与 `shrq $2` 的结果有何不同。

参考答案：1. `%rax` 是地址值 `0x100`，`(%rax)` 是该地址处的 8 字节内容 7。2. $0x200+3*8+16=0x228$ 。3. 可先执行 `movq (%rdi), %rax`，再执行 `movq %rax, (%rsi)`。4. 有效地址是 $x + 4x = 5x$ ，且 `leaq` 不解引用。5. 算术右移在高位补 1，保持负数符号；逻辑右移在高位补 0，得到很大的正数位模式。

7. 总结

- x86 通过长期演进和向后兼容形成了复杂但成功的 ISA；当前课程关注 x86-64。
- ISA 规定程序可见的行为，微体系结构决定这些行为如何由硬件实现。
- C 源码经过编译、汇编和链接成为可执行机器代码；汇编是机器代码的文本表示。
- 程序员可见状态包括 `%rip`、整数寄存器、条件码和按字节编址的内存。
- `movq` 在立即数、寄存器和内存之间传送 8 字节，但不能直接完成内存到内存复制。
- 地址表达式 `D(Rb,Ri,S)` 对应基址、索引、比例和位移；它既可用于访存，也可由 `leaq` 当作普通算术表达式使用。
- 阅读双操作数指令时要始终记住 AT&T 顺序：`opq Src, Dest`，结果写回目的操作数。